Module 4 - Threat Intelligence with Python

4.1 - Introduction to Threat Intelligence and Python Integration

1. What Is Threat Intelligence?

Threat Intelligence (TI) is contextual information about adversaries, their tactics, tools, and goals, used to inform and defend against cyber threats. It transforms raw indicators—like IPs or hashes—into meaningful insights.

TI can be categorized into:

- **Strategic**: High-level reports for executives, focusing on trends and risk.
- **Operational**: Details about ongoing campaigns, threat actors, and intent.
- **Tactical**: TTPs (Tactics, Techniques, Procedures) aligned with frameworks like MITRE ATT&CK.
- **Technical**: Raw IOCs (Indicators of Compromise): IPs, domains, file hashes, URLs, user-agents, registry keys.

Threat intelligence becomes actionable when we:

- Integrate it with detection systems (e.g., SIEMs, EDRs)
- Automate IOC enrichment and alert correlation
- Prioritize indicators based on confidence, severity, and timeliness

---

2. The Role of Python in Threat Intelligence

Python provides all necessary tools to:

- **Ingest** threat intelligence feeds (via HTTP, TAXII, APIs, local files)
- **Parse and normalize** structured formats like STIX, JSON, CSV

- **Enrich** indicators with context (reputation, geolocation, malware associations)
- **Correlate** IOCs with local logs, flows, alerts, or EDR data
- **Export** reports and dashboards in automated pipelines

Python is widely used in SOAR (Security Orchestration Automation and Response) platforms and TI tools due to its simplicity, large library ecosystem, and API compatibility.

---

3. Understanding TI Formats and Standards

A. STIX (Structured Threat Information Expression)

STIX is an open standard (maintained by OASIS) for describing threat intelligence. It uses structured JSON-like schemas to define:

- `Indicator` (IOC)
- `Threat Actor`
- `Malware`
- `Attack Pattern`
- `Observed Data`
- `TTPs`

Each STIX object includes relationships and metadata.

Example (simplified STIX 2.1 IOC):

```
{
  "type": "indicator",
  "id": "indicator--1234",
  "pattern": "[ipv4-addr:value = '8.8.8.8']",
  "pattern_type": "stix",
  "valid_from": "2025-06-14T00:00:00Z",
```

```
  "labels": ["malicious-activity"]
}
```

B. TAXII (Trusted Automated Exchange of Intelligence Information)

TAXII is a transport protocol for sharing STIX objects between servers and clients. Python can interface with TAXII servers using libraries like:

- `cabby` (TAXII 1.x)
- `taxii2-client` (TAXII 2.x)

TAXII servers can be public (e.g., MISP) or internal.

C. JSON/CSV/XML-Based Feeds

Many open threat feeds like AbuseIPDB, AlienVault OTX, or URLhaus offer raw data via:

- JSON APIs
- CSV dumps (updated daily)
- Direct downloads or authenticated access

Example (CSV from URLhaus):

```
dateadded,url,threat
2025-06-14,https://malicious[.]com,malware_download
```

---

4. Setting Up Your Python Environment

To work with Threat Intelligence in Python, ensure the following libraries:

```
pip install requests pandas stix2 taxii2-client python-misp
```

- `requests` – HTTP communication
- `pandas` – CSV/JSON parsing and manipulation
- `stix2` – STIX 2.1 object creation and parsing
- `taxii2-client` – Access TAXII 2.x feeds
- `python-misp` – Integration with MISP instance

---

5. Basic Feed Download with Python

Here is a basic example of pulling a JSON feed:

```python
import requests

feed_url = "https://otx.alienvault.com/api/v1/indicators/export?type=IPv4"
headers = {"X-OTX-API-KEY": "your_api_key"}

response = requests.get(feed_url, headers=headers)

if response.status_code == 200:
    with open("otx_ipv4_iocs.txt", "w") as f:
        f.write(response.text)
else:
    print("Failed to fetch feed:", response.status_code)
```

Comments:

- We request the AlienVault OTX feed of IPv4 IOCs.
- You must authenticate with an API key.
- Data is written to a local file for future parsing and correlation.

---

6. Parsing STIX Data in Python

If you receive a STIX 2.1 bundle:

```
from stix2 import parse

with open("example_bundle.json") as f:
    bundle = parse(f.read(), allow_custom=True)

for obj in bundle.objects:
    if obj.type == "indicator":
        print(f"[+] Indicator: {obj.pattern}")
```

Commentary:

- `parse()` auto-detects STIX object types.
- Each object has `.type`, `.pattern`, and other attributes.
- This allows you to filter only relevant objects (e.g., `indicator`, `malware`, etc.)

---

7. Use Case: Ingest + Enrich + Print Summary

Combining ingestion with simple enrichment:

```python
import requests
import json

# Step 1: Fetch IOCs (fake example)
response = requests.get("https://example.com/iocs.json")
iocs = response.json()

# Step 2: Simple enrichment via IP geolocation
def enrich_ip(ip):
    r = requests.get(f"https://ipinfo.io/{ip}/json")
    if r.status_code == 200:
        return r.json().get("country", "Unknown")
    return "Unknown"

# Step 3: Display enriched results
for entry in iocs.get("ips", []):
    country = enrich_ip(entry)
    print(f"[{entry}] → Located in {country}")
```

---

8. Summary

Concept	What You Learned
Threat Intelligence Types	Strategic, Operational, Tactical, Technical
STIX & TAXII	How structured threat data is defined and shared
Python Role	Ingesting, parsing, enriching, correlating, reporting
APIs and Formats	How to consume data from AlienVault OTX, AbuseIPDB, MISP, etc.
Setup Environment	Key Python libraries and tools

Concept	What You Learned
Practical Scripts	How to download, parse, and enrich IOCs in real-time

4.2 - Consuming Threat Intelligence Feeds

Overview

Consuming threat intelligence feeds is one of the foundational capabilities of modern security automation. Whether it's for hunting, detection tuning, IOC correlation, or enrichment, building a robust feed ingestion pipeline is critical.

This topic covers:

- Feed types (structured, unstructured, real-time, bulk)
- Common feed sources: STIX/TAXII, MISP, OTX, URLhaus
- Libraries and tools for consumption
- Writing resilient ingestion pipelines
- Filtering and normalizing feed data

---

1. Feed Types: Formats and Delivery Models

Threat feeds vary widely depending on provider and purpose. Here's a breakdown:

By Format

Format	Description
STIX	Structured JSON-based model (objects like Indicator, Malware, etc.)
CSV	Common for large bulk lists (URLhaus, EmergingThreats)

Format	Description
JSON	REST API responses (OTX, Greynoise)
MISP	JSON events with relationships (events, attributes, tags, sightings)

By Delivery

Delivery	Description
Pull	Periodically query and download feeds (CSV/JSON/REST)
Push	Webhooks or streaming APIs that push data to your service
TAXII	Protocol that serves STIX bundles via collections/endpoints

---

2. Setup: Python Libraries for Feed Ingestion

Install the following key tools:

```
pip install requests pandas stix2 taxii2-client python-misp
```

- `requests` : HTTP clients for pulling REST feeds
- `pandas` : CSV parsing and analysis
- `stix2` : Create/parse STIX indicators
- `taxii2-client` : Interact with TAXII servers
- `python-misp` : Query and parse data from MISP

---

3. Example 1: TAXII 2.0 Feed (STIX Pull)

Most commercial and open-source STIX feeds are served over TAXII.

```python
from taxii2client.v21 import Server

# Connect to a public TAXII server
server = Server("https://cti.taxii.org/taxii/")

# List API roots
api_root = server.api_roots[0]  # Usually default API root

# Discover collections
collections = api_root.collections
print("[+] Available Collections:")
for coll in collections:
    print(f"  - {coll.title}")

# Pull a bundle of objects from a collection
collection = collections[0]
objects = collection.get_objects()
for obj in objects['objects']:
    if obj['type'] == 'indicator':
        print(f"[!] Indicator: {obj['pattern']} (Labels: {obj['labels']})")
```

Explanation:

- Establishes a connection to a TAXII 2.0 server.
- Queries collections and retrieves STIX bundles.
- Filters objects of type `"indicator"` for printing.

---

4. Example 2: Ingesting CSV from URLhaus

CSV feeds are often updated hourly or daily.

```
import pandas as pd

url = "https://urlhaus.abuse.ch/downloads/csv/"
df = pd.read_csv(url, comment="#", encoding="utf-8")

# Filter recent malware URLs
malicious = df[df['threat'].notnull()][['dateadded', 'url', 'threat']]
print(malicious.head())
```

💡 **Notes**:

- `comment="#"` skips metadata headers.
- Easy filtering for specific `threat` types (e.g., ransomware, trojans).

---

5. Example 3: REST API (OTX by AlienVault)

AlienVault OTX provides API access with token authentication.

```
import requests

API_KEY = "your_otx_api_key"
headers = {"X-OTX-API-KEY": API_KEY}

r = requests.get("https://otx.alienvault.com/api/v1/pulses/subscribed", headers=headers)

if r.status_code == 200:
```

```
    data = r.json()
    for pulse in data["results"]:
        print(f"[+] Pulse: {pulse['name']}")
        for indicator in pulse['indicators']:
            print(f"    → {indicator['type']}: {indicator['indicator']}")
else:
    print("Failed to fetch OTX pulses.")
```

This script:

- Authenticates using your API key.
- Retrieves all subscribed threat “pulses.”
- Prints indicator type and values (IP, domain, hash, etc.).

---

6. Example 4: Querying a MISP Instance

MISP (Malware Information Sharing Platform) is a widely used open-source threat sharing platform.

```
from pymisp import ExpandedPyMISP

misp_url = "https://misp.example.com/"
misp_key = "your_api_key"

misp = ExpandedPyMISP(misp_url, misp_key, ssl=False)

# Search for events containing a specific IP
events = misp.search('attributes', value="8.8.8.8")

for event in events:
    print(f"[+] Event ID: {event['Event']['id']} - Info: {event['Event']['info']}")
```

📌 **Important**:

- Ensure `ssl=False` only in trusted lab/dev environments.
- You can also search by tag, attribute type (e.g., `md5`, `hostname`).

---

7. Best Practices for Feed Ingestion

Filter for Usefulness

- Ignore outdated indicators (older than 90 days).
- Prioritize by confidence, sightings, or verified actors.

Normalize the Structure

Use Python dicts to standardize different formats into a unified schema:

```
normalized = {
    "indicator": "1.1.1.1",
    "type": "ipv4-addr",
    "source": "AlienVault",
    "confidence": "high",
    "tags": ["botnet", "command-and-control"]
}
```

Schedule Regular Fetch

Use tools like `cron`, `schedule`, or `Celery` to automate periodic ingestion.

---

8. Outputting Ingested IOCs

Export parsed feeds to formats useful for SIEMs, SOARs, or dashboards.

```
import json

with open("output_iocs.json", "w") as f:
    json.dump(ioc_list, f, indent=2)
```

Or write to CSV:

```
import csv

with open("output.csv", "w", newline='') as f:
    writer = csv.DictWriter(f, fieldnames=["indicator", "type", "source"])
    writer.writeheader()
    for item in ioc_list:
        writer.writerow(item)
```

Summary

Feature	Example Tool/API	Format	Python Support
STIX/TAXII	MITRE CTI, EclecticIQ	STIX 2	taxii2-client, stix2
REST API	AlienVault OTX, Greynoise	JSON	requests
CSV Download	URLhaus, ET Pro Rules	CSV	pandas
Private TI Platform	MISP	JSON	python-misp

4.3 - Enriching IOCs via APIs

Overview

In cybersecurity, raw indicators such as IP addresses, domain names, URLs, and file hashes have limited value unless enriched with contextual metadata. Enrichment is the process of querying external data sources or intelligence providers to gather additional context such as reputation, historical activity, malware associations, geolocation, ASN ownership, and more.

This topic focuses on practical methods for enriching Indicators of Compromise (IOCs) using Python and publicly available APIs, along with proper techniques to normalize and aggregate that data for decision-making, automation, or alert triage.

Common IOC Types

- IP Addresses (IPv4/IPv6)
- Domain Names and Hostnames
- URLs
- File Hashes (MD5, SHA1, SHA256)

Key Goals of Enrichment

- Add risk/reputation scores
- Discover associations with malware or threat actors
- Retrieve passive DNS and resolution history
- Confirm presence in sandbox detonations
- Determine ASN, country, and network owner

---

1. Enrichment Services and APIs

API Provider	IOC Types	Free Access	Output Format
VirusTotal	Hashes, IPs, URLs	Limited	JSON
AbuseIPDB	IPs	Yes	JSON
Greynoise	IPs	Limited	JSON
Shodan	IPs, Domains	Yes	JSON
IPinfo / IPAPI	IPs	Yes	JSON
WhoisXML / RDAP	Domains, IPs	Partial	JSON/XML

---

2. Enriching IP Address Reputation: AbuseIPDB

AbuseIPDB provides information about whether an IP has been reported for abuse, fraud, botnets, or spam.

```
import requests

API_KEY = "your_abuseipdb_key"
ip = "45.33.32.156"

url = "https://api.abuseipdb.com/api/v2/check"
params = {
    "ipAddress": ip,
    "maxAgeInDays": 90
}
headers = {
    "Accept": "application/json",
    "Key": API_KEY
}
```

```
response = requests.get(url, headers=headers, params=params)
data = response.json()["data"]

# Print useful fields
print("IP:", data["ipAddress"])
print("Abuse Confidence Score:", data["abuseConfidenceScore"])
print("Country:", data["countryCode"])
print("Usage Type:", data["usageType"])
print("Domain:", data["domain"])
```

This provides insight into the IP's risk level and its hosting behavior.

---

3. Enriching File Hashes: VirusTotal API

VirusTotal is a well-known platform that aggregates AV scan results, sandbox data, and file metadata.

```
import requests

API_KEY = "your_virustotal_key"
hash_value = "44d88612fea8a8f36de82e1278abb02f"  # EICAR test file hash

headers = {
    "x-apikey": API_KEY
}
url = f"https://www.virustotal.com/api/v3/files/{hash_value}"

response = requests.get(url, headers=headers)
if response.status_code == 200:
    data = response.json()
    print("Malicious Detections:", data['data']['attributes']['last_analysis_stats']['malicious'])
    print("File Type:", data['data']['attributes']['type_description'])
```

```
    print("Meaningful Name:", data['data']['attributes'].get("meaningful_name"))
else:
    print("Not found or quota exceeded.")
```

Always obey rate limits and privacy guidelines with VirusTotal.

---

4. Enriching Domains: Passive DNS using SecurityTrails

```
import requests

API_KEY = "your_securitytrails_key"
domain = "example.com"
url = f"https://api.securitytrails.com/v1/domain/{domain}/subdomains"

headers = {
    "APIKEY": API_KEY
}

r = requests.get(url, headers=headers)
data = r.json()

print("Subdomains discovered:")
for subdomain in data["subdomains"]:
    print(f" - {subdomain}.{domain}")
```

This API gives visibility into infrastructure potentially related to the target domain.

---

5. Greynoise IP Context API

Greynoise reveals if an IP has been observed scanning or engaging in malicious behavior.

```
import requests

API_KEY = "your_greynoise_key"
ip = "45.33.32.156"

headers = {
    "key": API_KEY,
    "accept": "application/json"
}
url = f"https://api.greynoise.io/v3/community/{ip}"

r = requests.get(url, headers=headers)
data = r.json()

print("Classification:", data["classification"])
print("Name:", data["name"])
print("Link:", data["link"])
```

This is helpful for rapid triage of scanning IPs.

---

6. ASN and Geolocation Enrichment: IPinfo

```
import requests

ip = "8.8.8.8"
response = requests.get(f"https://ipinfo.io/{ip}/json")
data = response.json()
```

```
print("IP:", data["ip"])
print("City:", data.get("city"))
print("Region:", data.get("region"))
print("Country:", data.get("country"))
print("ASN:", data.get("org"))
```

Provides ownership, location, and usage data which can indicate infrastructure patterns.

---

7. Normalization of Enrichment Output

Because each API returns different structures, standardize them:

```
enriched = {
    "ioc": "8.8.8.8",
    "type": "ip",
    "abuse_score": 0,
    "greynoise_classification": "benign",
    "country": "US",
    "asn": "Google LLC",
    "virustotal_malicious": 0,
    "sources": ["abuseipdb", "greynoise", "ipinfo"]
}
```

You can wrap this into a function to handle multiple IOCs and outputs.

---

8. Combining Multiple Enrichments for a Single IOC

```python
def enrich_ip(ip):
    results = {"ioc": ip, "type": "ip"}

    # AbuseIPDB
    try:
        r = requests.get("https://api.abuseipdb.com/api/v2/check", params={"ipAddress": ip, "maxAgeInDays": 90},
                         headers={"Accept": "application/json", "Key": API_KEY})
        results["abuse_score"] = r.json()["data"]["abuseConfidenceScore"]
    except:
        results["abuse_score"] = None

    # IPinfo
    try:
        r = requests.get(f"https://ipinfo.io/{ip}/json")
        j = r.json()
        results["country"] = j.get("country")
        results["asn"] = j.get("org")
    except:
        results["country"] = results["asn"] = None

    return results
```

This approach abstracts the complexity behind reusable logic for enrichment.

---

9. Exporting Enriched Data

Export the enriched indicators to CSV or JSON for SIEM ingestion or dashboarding.

```python
import json
```

```python
with open("enriched_iocs.json", "w") as f:
    json.dump([enriched], f, indent=2)
```

Or to CSV:

```python
import csv

with open("enriched_iocs.csv", "w", newline='') as f:
    writer = csv.DictWriter(f, fieldnames=enriched.keys())
    writer.writeheader()
    writer.writerow(enriched)
```

---

10. Best Practices

- Always respect API rate limits and authentication rules.
- Build retry and caching logic for failed lookups or repeated IOCs.
- Filter by last seen dates or confidence scores to avoid noisy data.
- Aggregate multiple sources to improve reliability.
- Normalize and tag all results for SIEM correlation.

Summary

IOC enrichment is a vital step in turning raw threat data into actionable intelligence. Through structured queries to APIs like VirusTotal, AbuseIPDB, Greynoise, and others, you can programmatically gather relevant context for alerts, investigations, or automated responses. By standardizing, normalizing, and exporting these enriched results, you enhance your organization's visibility and decision-making capabilities.

4.4 - Generating Actionable Reports and Indicators

Overview

After enriching threat data such as IOCs (Indicators of Compromise), the final and critical step is to **translate that data into actionable intelligence**. This involves organizing and presenting the enriched results in a way that allows security analysts, SOC teams, and automated systems to quickly understand and act upon threats.

This topic will guide you through building **structured, readable, and consumable reports** in various formats (JSON, CSV, Excel, HTML, Markdown, PDF), embedding **actionable indicators**, prioritization flags, tags, and suggested responses.

---

1. Goals of Actionable Reporting

- **Clarity:** Remove ambiguity by presenting normalized, relevant data.
- **Priority:** Classify threats based on confidence, severity, or score.
- **Context:** Show relationships between indicators and known threats.
- **Automation:** Export to formats compatible with SIEMs, SOARs, or ticketing systems.

---

2. Structuring the Report Content

Core Sections in a Threat Report:

Section	Description
IOC Summary	Basic data and type of IOC
Enrichment Results	Metadata from services like VT, OTX, etc.
Threat Classification	Based on enrichment, severity, tags

Section	Description
Observables	Related domains, IPs, hashes
TTPs and MITRE Tags	Behavioral and contextual mapping
Recommendations	Actionable response items
Export Options	JSON, CSV, HTML, PDF, etc.

---

3. Creating a Normalized IOC Report Template in Python

```
def build_report(ioc_data):
    return {
        "ioc": ioc_data.get("ioc"),
        "type": ioc_data.get("type"),
        "source": ioc_data.get("sources", []),
        "reputation": ioc_data.get("abuse_score", 0),
        "classification": classify_threat(ioc_data),
        "geolocation": {
            "country": ioc_data.get("country"),
            "asn": ioc_data.get("asn")
        },
        "vt_malicious": ioc_data.get("virustotal_malicious", 0),
        "related_entities": ioc_data.get("related", []),
        "mitre_ttps": ioc_data.get("mitre", []),
        "recommendations": generate_response(ioc_data)
    }

def classify_threat(ioc):
    score = ioc.get("abuse_score", 0)
    if score >= 85 or ioc.get("vt_malicious", 0) > 10:
```

```
        return "High"
    elif score >= 50:
        return "Medium"
    else:
        return "Low"

def generate_response(ioc):
    if classify_threat(ioc) == "High":
        return ["Block IOC at perimeter", "Isolate affected asset", "Create SIEM rule"]
    elif classify_threat(ioc) == "Medium":
        return ["Monitor activity", "Flag IOC in threat feeds"]
    else:
        return ["No action required", "Log for future correlation"]
```

---

4. Exporting Reports in JSON and CSV

Export to JSON

```
import json

report = build_report(enriched_ioc)

with open("ioc_report.json", "w") as f:
    json.dump(report, f, indent=2)
```

Export to CSV

```
import csv

with open("ioc_report.csv", "w", newline='') as f:
```

```
    writer = csv.DictWriter(f, fieldnames=report.keys())
    writer.writeheader()
    writer.writerow(report)
```

---

5. Exporting to Excel using `openpyxl`

```
from openpyxl import Workbook

wb = Workbook()
ws = wb.active
ws.title = "IOC Report"

for i, (k, v) in enumerate(report.items(), start=1):
    ws[f"A{i}"] = k
    ws[f"B{i}"] = str(v)

wb.save("ioc_report.xlsx")
```

Excel is useful for sharing reports with non-technical stakeholders.

---

6. Exporting to Markdown and HTML

Markdown Output

```
md = f"""# IOC Report: {report['ioc']}

**Type:** {report['type']}
**Reputation Score:** {report['reputation']}
```

```
**Country:** {report['geolocation']['country']}
**ASN:** {report['geolocation']['asn']}
**VirusTotal Detections:** {report['vt_malicious']}
**Classification:** {report['classification']}

## Recommendations
"""
for rec in report["recommendations"]:
    md += f"- {rec}\n"

with open("ioc_report.md", "w") as f:
    f.write(md)
```

Convert Markdown to HTML using `markdown` library:

```
import markdown

html = markdown.markdown(md)
with open("ioc_report.html", "w") as f:
    f.write(html)
```

---

7. Exporting to PDF

Use `reportlab` or convert HTML to PDF with `pdfkit` and `wkhtmltopdf`.

```
import pdfkit

pdfkit.from_file("ioc_report.html", "ioc_report.pdf")
```

---

8. Grouping and Aggregating Multiple IOC Reports

```
all_reports = []

for ioc in ioc_list:
    enriched = enrich_ip(ioc)
    rep = build_report(enriched)
    all_reports.append(rep)

with open("all_ioc_reports.json", "w") as f:
    json.dump(all_reports, f, indent=2)
```

You can group IOCs by severity or type before exporting.

---

9. Including MITRE ATT&CK Tags

Add TTPs based on API detection (e.g., phishing -> T1566, command and control -> T1071):

```
def tag_mitre(ioc):
    if "phishing" in ioc.get("related_tags", []):
        return ["T1566"]
    if "c2" in ioc.get("behavior", ""):
        return ["T1071"]
    return []
```

This helps align with MITRE frameworks in SIEMs or reports.

---

10. Delivering Actionable Intelligence

Ensure that each report answers these questions:

1. **What is the IOC?**
2. **How dangerous is it?**
3. **What is it associated with?**
4. **What should we do about it?**

Make your report readable by both humans and machines.

---

11. Sending Reports via Email (Optional)

```
import smtplib
from email.message import EmailMessage

msg = EmailMessage()
msg["Subject"] = "IOC Alert Report"
msg["From"] = "analyst@example.com"
msg["To"] = "soc@example.com"
msg.set_content(md)

with open("ioc_report.xlsx", "rb") as f:
    msg.add_attachment(f.read(), maintype="application", subtype="xlsx", filename="ioc_report.xlsx")

s = smtplib.SMTP("smtp.example.com")
s.send_message(msg)
s.quit()
```

Summary

Creating actionable threat reports is critical for transforming enriched threat data into decisions. Using Python, you can automate the full workflow—from enrichment to report generation and export in multiple formats. This enables SOCs and analysts to act faster, triage threats better, and provide high-value intel to decision-makers.